

# Documentazione - Analisi dei requisiti

## Documentazione Prodotto Software: SmartLog

---

### 1. Analisi dei Requisiti

#### 1.1. Introduzione

Il progetto **SmartLog** mira a fornire agli utenti una piattaforma semplice e intuitiva per tracciare i propri allenamenti di fitness. L'applicazione permette di registrare gli allenamenti, specificando la data, e di aggiungere dettagli per ogni esercizio eseguito, come nome dell'esercizio, peso (kg), ripetizioni e note opzionali.

#### 1.2. Requisiti Funzionali

- **RF1: Autenticazione Utente**

- **RF1.1:** Il sistema deve permettere a un nuovo utente di registrarsi fornendo nome (obbligatorio), secondo nome (opzionale), cognome (obbligatorio), secondo cognome (opzionale), email e password.
- **RF1.2:** Il sistema deve permettere a un utente registrato di effettuare il login tramite email e password.
- **RF1.3:** Il sistema deve gestire il logout dell'utente.

- **RF2: Gestione Allenamenti**

- **RF2.1:** Il sistema deve permettere a un utente autenticato di visualizzare la lista dei propri allenamenti passati, ordinati per data.
- **RF2.2:** Il sistema deve permettere a un utente autenticato di aggiungere un nuovo allenamento, specificando la data.
- **RF2.3:** Il sistema deve permettere a un utente autenticato di modificare la data di un allenamento esistente.

- **RF2.4:** Il sistema deve permettere a un utente autenticato di eliminare un allenamento esistente, comprese tutte le esecuzioni ad esso associate.
- **RF3: Gestione Esecuzioni (Esercizi)**
  - **RF3.1:** Per ogni allenamento, il sistema deve permettere all'utente di aggiungere una o più esecuzioni di esercizi.
  - **RF3.2:** Per ogni esecuzione, l'utente deve poter specificare il nome dell'esercizio, i kg sollevati e il numero di ripetizioni.
  - **RF3.3:** Per ogni esecuzione, l'utente deve poter aggiungere note testuali opzionali.
  - **RF3.4:** Il sistema deve permettere all'utente di modificare i dettagli di un'esecuzione esistente (nome esercizio, kg, ripetizioni, note).
  - **RF3.5:** Il sistema deve permettere all'utente di eliminare un'esecuzione esistente da un allenamento.

### 1.3. Requisiti Non Funzionali

- **RNF1: Usabilità:** L'interfaccia utente deve essere intuitiva e facile da usare. Il widget `Password` con toggle di visibilità contribuisce a questo.
- **RNF2: Performance:** Le operazioni di caricamento dati e interazione con il backend devono essere ragionevolmente veloci.
- **RNF3: Sicurezza (Backend):** La logica di backend deve gestire la connessione al database in modo sicuro e validare gli input. Le password, sebbene attualmente memorizzate in chiaro (come indicato nel codice PHP), dovrebbero idealmente essere hashate.
- **RNF4: Portabilità (Frontend):** L'applicazione frontend è sviluppata in Flutter, il che suggerisce l'obiettivo di poter essere eseguita su diverse piattaforme mobili.
- **RNF5: Serializzazione Dati:** I modelli di dati del frontend devono poter essere serializzati e deserializzati da/verso JSON per la comunicazione con il backend, gestita tramite `json_serializable`.

### 1.4. Attori

- **Utente Registrato:** Persona che utilizza l'applicazione per tracciare i propri allenamenti.
- 

## 2. Relazione Finale sul Prodotto Realizzato

### 2.1. Architettura Generale

Il sistema SmartLog è composto da due componenti principali:

- **Applicazione Frontend Mobile:** Sviluppata utilizzando il framework Flutter. Questa si occupa della presentazione dell'interfaccia utente, della gestione dei dati localmente tramite modelli e della comunicazione con il backend.
- **API Backend:** Realizzata in PHP, gestisce la logica di business, l'interazione con il database MySQL (denominato `smartlog`) e risponde alle richieste del client Flutter in formato JSON.

### 2.2. Descrizione dei Moduli Frontend (Flutter)

- `main.dart` :
  - Punto di ingresso dell'applicazione Flutter.
  - Imposta il tema globale dell'applicazione, incluso il font `BebasNeue`.
  - Definisce `LoginPage` come schermata iniziale.
- `signup.dart` (**Schermata di Registrazione**):
  - Fornisce un'interfaccia utente per la registrazione di nuovi utenti.
  - Include campi per nome, secondo nome (opzionale), cognome, secondo cognome (opzionale), email e password. Il campo password utilizza il widget custom `Password`.
  - Utilizza `TextEditingController` per gestire l'input dell'utente.
  - Interagisce con `ApiClient` per inviare i dati di registrazione al backend.
  - In caso di successo, reindirizza l'utente alla `LoginPage`.
  - Mostra messaggi di errore tramite `SnackBar` in caso di fallimento.
  - Presenta un logo SVG e un link per navigare alla pagina di login se l'utente ha già un account.

- **login.dart (Schermata di Login):**
  - Fornisce un'interfaccia utente per l'autenticazione degli utenti esistenti.
  - Include campi per email e password, quest'ultima gestita dal widget `Password`.
  - Utilizza `TextEditingController` per l'input.
  - Interagisce con `ApiClient` per inviare le credenziali al backend.
  - In caso di successo, recupera i dati dell'utente e naviga alla `HomePage`, passando l'oggetto `User`.
  - Mostra messaggi di errore tramite `SnackBar`.
  - Presenta un logo SVG e un link per navigare alla pagina di registrazione.
- **home.dart (Schermata Principale):**
  - Visualizza un messaggio di benvenuto personalizzato per l'utente loggato (utilizzando `widget.user.name1`).
  - Permette di visualizzare la lista degli allenamenti dell'utente.
  - **Funzionalità Allenamenti:**
    - Carica e mostra gli allenamenti esistenti recuperandoli tramite `ApiClient`.
    - Pulsante "Aggiungi allenamento": apre un `DatePicker` per selezionare la data del nuovo allenamento e invia la richiesta di creazione tramite `ApiClient`.
    - Per ogni allenamento ( `Allenamento` ) visualizzato in una `Card` :
      - Mostra la data dell'allenamento.
      - Un `PopupMenuButton` (icona con tre puntini) permette di:
        - "Modifica Allenamento": apre un `DatePicker` per cambiare la data.
        - "Elimina Allenamento": chiede conferma e poi procede all'eliminazione tramite `ApiClient`.
      - Un `IconButton` (icona '+') permette di aggiungere un'esecuzione ( `Esecuzione` ) all'allenamento.
  - **Funzionalità Esecuzioni:**

- Visualizza le esecuzioni (esercizi) associate a ciascun allenamento, mostrando nome esercizio, kg, ripetizioni e note.
- "Aggiungi esecuzione": Apre un `AlertDialog` con campi per nome esercizio, kg, ripetizioni e note. Invia i dati tramite `ApiClient`.
- Per ogni esecuzione:
  - Un `PopupMenuButton` permette di:
    - "Modifica Esecuzione": apre un `AlertDialog` precompilato per modificare i dettagli dell'esecuzione.
    - "Elimina Esecuzione": chiede conferma e poi procede all'eliminazione tramite `ApiClient`.
  - Include un pulsante di logout che riporta alla `LoginPage`.
  - Utilizza `SvgPicture` per visualizzare loghi.
- **`api_client.dart`**:
  - Classe responsabile della comunicazione HTTP con il backend PHP.
  - Utilizza il package `http`.
  - URL base del backend: `http://localhost/progettotts/index_php.php`.
  - Metodi implementati per `signUp`, `login`, `WorkspaceAllenamenti`, `aggiungiAllenamento`, `aggiungiEsecuzione`, `modificaAllenamento`, `eliminaAllenamento`, `modificaEsecuzione`, `eliminaEsecuzione`.
  - Gestisce le risposte JSON e solleva eccezioni in caso di errori HTTP o logici dal backend.
- **Modelli Dati (nel direttorio `model`):**
  - **`user.dart`**: Definisce la classe `User` con proprietà come `userID`, `name1`, `name2` (opzionale), `surname1`, `surname2` (opzionale), `email`, `password` e una lista opzionale di `Sessione` (`sessioni`). Utilizza `@JsonSerializable()` per la generazione automatica del codice di serializzazione/deserializzazione (`user.g.dart`).
  - **`allenamento.dart`**: Definisce la classe `Allenamento` con `trainingID`, `data` (modificabile), una lista opzionale di `Esecuzione` (`esecuzioni`) e una lista opzionale di `Sessione` (`sessioni`). Utilizza `@JsonSerializable()` (`allenamento.g.dart`).

- **esecuzione.dart** : Definisce la classe `Esecuzione` con `executionID` , `kg` , `ripetizioni` , `note` (opzionale), `trainingID` e `nomeEsercizio` . Utilizza `@JsonSerializable()` ( `esecuzione.g.dart` ) e custom converters ( `StringToDoubleConverter` , `StringToIntConverter` ) per la gestione di tipi numerici che potrebbero arrivare come stringhe dal JSON.
- **sessione.dart** : Definisce la classe `Sessione` con `userID` e `trainingID` , rappresentando la relazione multi-a-molti tra utenti e allenamenti. Include riferimenti opzionali a `User` e `Allenamento` per evitare cicli di dipendenza nella serializzazione. Utilizza `@JsonSerializable()` ( `sessione.g.dart` ).
- **converters.dart** : Contiene `StringToDoubleConverter` e `StringToIntConverter` , classi che implementano `JsonConverter` per convertire in modo sicuro valori dinamici (stringhe o numeri) dal JSON ai tipi `double` e `int` rispettivamente, con gestione dei valori null o malformati.
- **Widget Personalizzati:**
  - **password.dart** : Definisce un widget `Password` che è uno `StatefulWidget` . Incapsula un `TextField` specificamente per l'input di password, con la funzionalità di oscurare/mostrare il testo tramite un `GestureDetector` che cambia l'icona ( `Icons.visibility_off` / `Icons.visibility` ).

## 2.3. Descrizione dei Moduli Backend (PHP)

- **index\_php.php** :
  - Punto di ingresso per tutte le richieste API dal client Flutter.
  - Imposta gli header per la risposta JSON e per il CORS.
  - Gestisce le richieste preflight `OPTIONS` .
  - Include `funzioni.inc` .
  - Determina il tipo di richiesta (GET o POST) e il parametro `request` per smistare l'azione alla funzione corretta.
  - Per le richieste POST, decodifica il corpo della richiesta JSON.
  - Richiama le funzioni appropriate per le varie operazioni CRUD su utenti, allenamenti ed esecuzioni.

- **funzioni.inc** :
  - Contiene la logica di business e l'interazione con il database.
  - **connetti()** : Stabilisce una connessione al database MySQL **smartlog** utilizzando PDO.
  - **ritornaErroreJSON()** : Invia risposte di errore JSON.
  - **login()** : Gestisce il login, confrontando la password in chiaro.
  - **registrazione()** : Gestisce la registrazione, salvando la password in chiaro.
  - **aggiungiAllenamento()** : Aggiunge un allenamento e una sessione, usando transazioni.
  - **aggiungiEsecuzione()** : Aggiunge un'esecuzione a un allenamento.
  - **getAllenamentiUtente()** : Recupera allenamenti ed esecuzioni per un utente, raggruppandoli.
  - **modificaAllenamento()** : Modifica la data di un allenamento.
  - **eliminaAllenamento()** : Elimina un allenamento (presumendo **ON DELETE CASCADE** ).
  - **modificaEsecuzione()** : Modifica i dettagli di un'esecuzione.
  - **eliminaEsecuzione()** : Elimina un'esecuzione.

## 2.4. Struttura del Database (Inferita e Confermata dai Modelli)

Basandosi sulle query in **funzioni.inc** e sui modelli dati Flutter, si conferma e si dettaglia la struttura delle tabelle:

- **users** :
  - **userID** (INT, PK, AI)
  - **name1** (VARCHAR)
  - **name2** (VARCHAR, NULLABLE)
  - **surname1** (VARCHAR)
  - **surname2** (VARCHAR, NULLABLE)
  - **email** (VARCHAR, UNIQUE)

- `password` (VARCHAR) - Attualmente memorizza password in chiaro
- **allenamenti** :
  - `trainingID` (INT, PK, AI)
  - `data` (DATE o DATETIME)
- **sessioni** :
  - `sessionID` (INT, PK, AI - presunto, o PK composta da `userID` , `trainingID` )
  - `userID` (INT, FK references `users.userID` )
  - `trainingID` (INT, FK references `allenamenti.trainingID` , presumibilmente con `ON DELETE CASCADE` )
- **esecuzioni** :
  - `executionID` (INT, PK, AI)
  - `nomeEsercizio` (VARCHAR)
  - `kg` (DECIMAL o FLOAT)
  - `ripetizioni` (INT)
  - `note` (TEXT o VARCHAR, NULLABLE)
  - `trainingID` (INT, FK references `allenamenti.trainingID` , presumibilmente con `ON DELETE CASCADE` )

## 2.5. Conclusioni e Possibili Miglioramenti

Il prodotto SmartLog realizzato soddisfa i requisiti funzionali di base.

L'introduzione di modelli dati strutturati con `json_serializable` e widget riutilizzabili come `Password` migliora la manutenibilità e la qualità del codice frontend.

Possibili aree di miglioramento (confermate dalla precedente analisi) includono:

- **Sicurezza Password:** Implementare l'hashing delle password nel backend PHP.
- **Validazione Input:** Rafforzare la validazione sia lato client che server.
- **Gestione degli Errori:** Migliorare la granularità dei messaggi di errore.
- **Interfaccia Utente:** Ulteriori rifiniture dell'UI/UX.



- **Funzionalità Avanzate:** Statistiche, grafici, ecc.
- **Test:** Implementare unit test e integration test.
- **Gestione Stato (Flutter):** Considerare soluzioni di state management più avanzate per future espansioni.
- **Documentazione API:** Formalizzare la documentazione API.
- **Deployment:** Definire e documentare il processo di deployment.

L'uso di `json_serializable` e dei `JsonConverter` personalizzati assicura una gestione robusta della (de)serializzazione dei dati tra frontend e backend.